

Design Document: RL-Driven Physical Activity Interventions

A Configurable Simulation Framework

William Dennis
Bristol x NUS Research Project
ky23812@bristol.ac.uk

June 2026

Contents

1	Problem Statement	3
1.1	Background	3
1.2	Problem	3
1.3	Goals	3
1.4	Scope	3
2	Design Requirements	4
2.1	Stakeholders	4
2.2	Constraints	4
2.3	Requirements	4
2.4	Non-Requirements	4
3	Architecture Overview	4
3.1	Design Principles	4
3.2	Component Overview	5
3.3	Data Pipeline	5
3.4	Module Layout	5
3.5	Configurability	6
3.6	Validation	6
3.7	Test Organisation	7
4	Decision Log	7
5	Open Decisions	7
5.1	Distribution model	7
5.2	MDP baseline approval	8
5.3	Phase 2 dataset targets	8
5.4	Simulator validation criteria	8
6	MDP Formalisation	8
6.1	State Space $\mathcal{S}$	8
6.2	Action Space $\mathcal{A}$	9
6.3	Transition Function P	9
6.4	Reward Function R	9
6.4.1	Immediate Reward	9
6.4.2	Delayed Reward (Body Measures)	10
6.5	Discount Factor γ	10

6.6	Decision Epoch	10
7	Simulator Training Method	10
7.1	Parameter Estimation from MRT Data	10
7.2	Calibration Loop	11
7.3	Fidelity Metrics	11
7.4	Limitations	11
8	Data Sources	11
8.1	Phase 1: Synthetic Data	11
8.2	Phase 2: Real Data Targets	11
8.3	Dataset Survey	12
9	Timeline & Future Work	12
9.1	Phase 1 Milestones	12
9.2	Phase 2: Future Work	12

1 Problem Statement

1.1 Background

Physical inactivity is a leading risk factor for chronic disease worldwide. Just-in-time adaptive interventions (JITAI) delivered through wearable devices aim to provide the right support at the right moment, leveraging real-time sensor data to trigger personalised prompts. However, designing effective intervention policies—deciding *when*, *what*, and *how much* to intervene—is a sequential decision-making problem that traditional rule-based approaches handle poorly.

Reinforcement learning provides a natural framework for this problem: an agent learns a policy that maps user state (step count, sleep, heart rate, time of day) to an intervention action (motivational prompt, goal reminder, rest suggestion), optimising long-term behaviour change rather than short-term response. The key challenge is that training RL agents requires either expensive real-world trials or a realistic simulation environment.

1.2 Problem

There is no standardised, configurable simulation platform for evaluating RL agents on PA intervention tasks. Existing work uses bespoke simulators tied to specific datasets or papers. This makes cross-paper comparison difficult and limits reproducibility. Five specific gaps in the current landscape motivate this work:

1. No LLM simulation validated for wearable health data
2. No delayed-feedback bandit for 3-week clinical measures
3. No non-stationary adaptation for PA interventions
4. No standardised MDP/benchmark for PA JITAI RL [1]
5. No offline RL for PA interventions with real wearable data

These gaps are *experiments run on the framework*, not part of the framework itself. The framework provides the infrastructure to investigate each one.

1.3 Goals

This project delivers:

1. A **config-first simulation framework** where the dataset schema, MDP, user behaviour, and agents are specified entirely via YAML config files.
2. A **configurable MDP environment** supporting multi-timescale rewards (daily steps + delayed clinical measures).
3. A **rule-based user simulator** grounded in behavioural theory, with configurable archetypes and fatigue mechanisms.
4. A **library of RL agents** sharing a common interface, from simple baselines (Thompson Sampling) to deep RL (DQN, PPO).
5. An **experiment runner** for reproducible multi-trial, multi-agent comparisons.

1.4 Scope

Phase 1 (8 weeks) delivers a working CLI: `uv run rl-health-interventions --config experiment.yml` that trains agents and produces a results table. Phase 2 (stretch) adds LLM-based user simulation and validation against real data.

2 Design Requirements

2.1 Stakeholders

- **William Dennis** – implementation, single developer
- **Mengyan Zhang** (Bristol) – domain expertise, data access, supervision
- **Swapnil Mishra** (NUS / MLGH) – supervision, downstream research team

2.2 Constraints

- 8-week internship timeline
- Single developer, no prior codebase
- No existing real data access (4-8 week institutional applications required)
- Deliverable must be usable by Swapnil’s team after handover

2.3 Requirements

R1 – Config-first

All components (dataset schema, MDP, user behaviour, agents, experiments) are specified via YAML config files. New dataset or intervention design requires a new config, not new code.

R2 – Reproducible

Fixed seeds, config snapshot captured per run, deterministic evaluation mode.

R3 – Pluggable

Each component family (transitions, rewards, agents, response models) uses an ABC + registry pattern. Adding a new implementation requires one file and one registry entry.

R4 – Minimal dependencies

Python 3.11, uv, Polars, Pydantic, NumPy. No Stable Baselines or Gymnasium unless proven necessary.

R5 – Production-quality error handling

Three-layer validation (schema, compatibility, dummy step). Per-episode exception isolation. Structured JSON trace output.

R6 – Single-entry experiment

`uv run rl-health-interventions --config experiment.yml` builds, runs, and reports. No multi-step workflow.

2.4 Non-Requirements

The following are explicitly out of scope for Phase 1:

- Web UI or dashboard
- Real data integration (All of Us, UK Biobank)
- Deep RL agents (DQN, PPO, CQL, IQL)
- LLM-based user simulation
- Gymnasium or SB3 compatibility layer
- Real-time or online learning

3 Architecture Overview

3.1 Design Principles

- **Config-first:** Every component reads from YAML. New dataset or intervention design = new config, not new code.

- **Pluggable components:** Each family (transitions, rewards, agents, response models) uses an ABC + registry pattern. Adding a new component requires one file, one registry entry, and a config string reference.
- **Minimal dependencies:** Python 3.11, uv, Polars, Pydantic, NumPy. No Stable Baselines or Gymnasium unless proven necessary.
- **Reproducibility:** Config snapshots, fixed seeds, deterministic evaluation.

3.2 Component Overview

The framework is organised into five subphases, each delivering a pluggable component:

1. **Data Layer (1A):** Ingests CSV/Parquet files via Polars lazy scans, applies config-driven feature engineering, and materialises a numpy-backed `Dataset`. Provides a `StateView` bridge for the simulation loop.
2. **MDP Environment (1B):** Exposes a clean `step(state, action) → (next_state, reward, done)` API. Pluggable `TransitionModel` and `RewardHandler` components are selected from a registry by config string.
3. **User Simulation (1C):** Rule-based behavioural archetypes (goal-driven, social responder, resistant, stable maintainer) with backlash and fatigue mechanisms. A `ResponseModel` maps (state, action, profile) to a next state.
4. **Agent Library (1D):** Common Agent ABC with `select_action(state) → int` and `update(...)`. Thompson Sampling is the initial implementation; deep RL agents (DQN, PPO, CQL, IQL) are stretch.
5. **Experiment Runner (1E):** Wires all components via a single `ExperimentFactory.build(config)`. Runs multi-trial, multi-agent experiments with config snapshot reproducibility.

3.3 Data Pipeline

```
Raw files (CSV, Parquet, ...)      <- researcher provides these
|
Polars LazyFrame scan              <- config-driven schema mapping
|
FeaturePipeline.from_config(config) <- config-driven transforms (agg, window, diff, etc.)
| (Polars lazy graph, materialised once)
numpy arrays                      <- materialised for sim loop
|
Dataset (user x time index)       <- random access for StateView slices
```

Polars owns the feature engineering stage. Its lazy API maps one-to-one with the config-driven specification—grouping, rolling windows, expressions. The framework builds a `pl.LazyFrame` from the config and materialises it once. The result is converted to numpy arrays for the simulation loop, which needs fast random access by user and time rather than query capability.

After materialisation, the simulation depends only on numpy—no Polars dependency in the hot path.

3.4 Module Layout

Each pluggable concept gets a flat package with a `_base.py` ABC and a `REGISTRY` dict in `__init__.py`:

```
transitions/          <- pluggable transition models
  _base.py            <- TransitionModel ABC
  rule_based.py       <- RuleBasedTransition
```

```

__init__.py      <- imports + REGISTRY dict
rewards/         <- pluggable reward functions
  _base.py       <- RewardHandler ABC
  compound.py    <- CompoundReward
  __init__.py    <- imports + REGISTRY dict
agents/         <- pluggable agents
  _base.py       <- Agent ABC
  thompson_sampling.py <- ThompsonSamplingAgent
  __init__.py    <- imports + REGISTRY dict
simulation/      <- pluggable user response models
  _base.py       <- ResponseModel ABC
  rule_based.py  <- RuleBasedResponse
  __init__.py    <- imports + REGISTRY dict
data/           <- data layer
  _base.py       <- IngestPipeline ABC
  feature_pipeline.py <- Config-driven feature engineering on Polars
  polars_reader.py <- Reads CSV/Parquet via Polars lazy API
  dataset.py     <- Dataset (numpy arrays, for sim loop)
  synthetic.py   <- SyntheticDataGenerator
  __init__.py    <- imports + REGISTRY dict

```

Rules:

- `_base.py` contains only the ABC. No implementations.
- Each implementation is its own file, named after the class in snake_case.
- `__init__.py` explicitly imports every implementation and populates a `REGISTRY` dict. No auto-discovery.

3.5 Configurability

All MDP components are exposed in the config schema:

Table 1: Configurable parameters

Category	Parameter	Default
State	Variable selection	All
Actions	Available subsets	$\{a_0, \dots, a_5\}$
Reward	$(\alpha, \beta, \lambda, \eta)$	(1.0, 0.1, 0.5, 1.0)
Transition	Archetype	goal-driven
Fatigue	$B_{\max}$	3
Discount	γ	0.95
Epoch	Frequency	Daily

3.6 Validation

The factory applies three layers of validation before the main experiment loop:

1. **Schema validation (Pydantic):** Malformed configs are rejected at load time with a precise error message.
2. **Component compatibility:** Registry keys and cross-references are checked before instantiation.
3. **Dummy step:** A single `step()` call catches wiring errors and shape mismatches before the main loop.

3.7 Test Organisation

```
tests/
  unit/
    transitions/      <- each transition model in isolation
    rewards/         <- each reward function in isolation
    agents/          <- each agent in isolation
    simulation/       <- each response model in isolation
    factory/         <- factory validation and wiring
  integration/
    test_dummy_step.py <- factory builds and step succeeds
    test_config.yml   <- minimal config is loadable
  end_to_end/
    test_experiment.py <- full pipeline: config -> build -> train -> results
```

Rules:

- Every ABC method has at least one test with a real implementation.
- Every subphase gate requires all tests in that subphase’s modules.
- Test structure maps to modules, not subphase numbers.
- Registry discovery test ensures every registered component can be instantiated from a minimal config.

4 Decision Log

Table 2: Design decisions

#	Decision	Status
1	Config format: YAML	Confirmed
2	Interface: CLI + config files; web UI = stretch	Confirmed
3	Python 3.11 + uv for package management	Confirmed
4	Synthetic data for Phase 1 (real data requires 4-8 week applications)	Confirmed
5	Thompson Sampling as initial agent	Confirmed
6	Daily decision epochs	Confirmed
7	Multi-timescale reward: immediate (daily steps) + delayed (3-week body measures)	Confirmed
8	Four behavioural archetypes: goal-driven, social responder, resistant, stable maintainer	Confirmed
9	Pluggable component architecture (ABC + registry pattern)	Confirmed
10	Minimal dependencies: no Gymnasium, no SB3	Confirmed

5 Open Decisions

The following decisions require supervisor input before they affect implementation direction.

5.1 Distribution model

Should the framework be distributed as an installable package or remain repo-based?

Option A – Installable package

uv build produces a wheel. Researchers run `uv add rl-health-interventions` in their own project. Clearer dependency boundary, but requires CI/CD for publishing.

Option B – Repo-based

Clone and uv sync. Simpler for a single-team prototype. Swapnil’s team would fork or clone directly.

5.2 MDP baseline approval

Does the proposed MDP formalisation (§6) look like a reasonable starting point? The state space, action set, and reward function are intentionally conservative—they can be extended via config without breaking changes.

5.3 Phase 2 dataset targets

Which datasets should we target for real-data validation?

A recommended order: HeartSteps first (fastest access, most relevant), then All of Us (largest population), then UK Biobank (raw accelerometer).

5.4 Simulator validation criteria

How do we determine the simulator is “good enough” before running experiments? Candidate metrics include response-rate RMSE against real MRT data, state-distribution KL divergence, and engagement-curve alignment. We need agreement on acceptable thresholds.

6 MDP Formalisation

This section specifies the Markov Decision Process (MDP) for the first iteration of the simulation framework. The MDP is defined as the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$.

6.1 State Space $\mathcal{S}$

The state represents the current context of a user at decision epoch t . States are constructed from wearable-device signals, user profile information, and intervention history:

$$s_t = \{\mathbf{x}_t^{\text{wearable}}, \mathbf{x}_t^{\text{context}}, \mathbf{x}_t^{\text{history}}, \mathbf{x}^{\text{profile}}\}$$

Table 3: State variables

Variable	Type	Description
steps _t	$\mathbb{R}_{\geq 0}$	Step count, past 24h
hr _t	$\mathbb{R}_{\geq 0}$	Heart rate (or resting)
sleep_hours _t	$\mathbb{R}_{\geq 0}$	Previous night sleep
sedentary_min _t	$\mathbb{R}_{\geq 0}$	Sedentary minutes today
time_of_day _t	$\{0, \dots, 23\}$	Current hour
day_of_week _t	$\{0, \dots, 6\}$	Day of week
goal_progress _t	$[0, 1]$	Fraction of daily goal met
burden _t	$\mathbb{Z}_{\geq 0}$	Interventions sent today
a _{t-1}	$\mathcal{A}$	Previous action
response _{t-1}	$\{0, 1\}$	Previous response
body_measure _k	$\mathbb{R}$	Clinical measure (q3wks)
age	$\mathbb{Z}_{> 0}$	Static profile
gender	$\{0, 1\}$	Static profile
baseline_activity	$\{0, 1, 2\}$	Low / medium / high

Variables are normalised to $[0, 1]$ before being passed to the agent. The body measure is only observed every 3 weeks (sparse reward signal).

6.2 Action Space $\mathcal{A}$

$$\mathcal{A} = \{a_0, a_1, a_2, a_3, a_4, a_5\}$$

Table 4: Action space

Action	Label	Description
a_0	No message	Do not intervene
a_1	Motivational prompt	Generic encouragement
a_2	Walking suggestion	Recommend a short walk
a_3	Goal reminder	Remind of step goal
a_4	Recovery suggestion	Stretch or rest
a_5	Progress feedback	Report goal progress

Exactly one action per decision epoch. The “no message” action is critical for managing notification burden.

6.3 Transition Function P

The transition function models the probability over next states:

$$P(s_{t+1} \mid s_t, a_t) : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$$

In Phase 1 (rule-based), transitions follow a behavioural response model:

$$\Delta \text{steps}_{t+1} = f(s_t, a_t, \theta_{\text{user}})$$

where θ_{user} parameterises one of four behavioural archetypes:

- **Goal-driven:** Responds to reminders and feedback. Proportional to goal progress.
- **Social responder:** Responds to motivational prompts. Models social desirability bias.
- **Resistant:** Low response. Fatigue accumulates quickly.
- **Stable maintainer:** Already active. Low marginal gain.

Burden evolves as:

$$\text{burden}_{t+1} = \begin{cases} \text{burden}_t + 1 & \text{if } a_t \neq a_0 \\ 0 & \text{if } a_t = a_0 \end{cases}$$

When $\text{burden} > B_{\text{max}}$, response probability decays linearly.

6.4 Reward Function R

6.4.1 Immediate Reward

$$R_t^{\text{immediate}} = \alpha \cdot \Delta \text{steps}_t - \beta \cdot \mathbf{1}_{\{a_t \neq a_0\}} + \lambda \cdot \text{goal_progress}_t$$

6.4.2 Delayed Reward (Body Measures)

Clinical body measures arrive every 3 weeks ($t = 3k$):

$$R_{3k}^{\text{delayed}} = \eta \cdot \text{BM_improvement}_{3k}$$

Total reward:

$$R_t = \begin{cases} R_t^{\text{immediate}} + R_t^{\text{delayed}} & t \equiv 0 \pmod{3} \\ R_t^{\text{immediate}} & \text{otherwise} \end{cases}$$

The compound reward (immediate + delayed) is a central research question. Weights $(\alpha, \beta, \lambda, \eta)$ are configurable experiment parameters.

6.5 Discount Factor γ

$$\gamma \in [0.9, 0.95]$$

A high discount factor encourages long-term behaviour change over short-term step increases. Appropriate for health habit formation.

6.6 Decision Epoch

Daily decisions. This aligns with typical intervention cadence, the 3-week body measure delay (3 epochs), and practical deployment constraints.

7 Simulator Training Method

The user simulator (1C) transforms a current state and chosen action into a next state via a parameterised response model:

$$s_{t+1} = \text{ResponseModel}(s_t, a_t; \theta)$$

This section describes how θ is estimated and validated.

7.1 Parameter Estimation from MRT Data

Micro-randomized trials (HeartSteps V1/V2) provide sequences of (s_t, a_t, s_{t+1}) tuples. For each user, we observe how their step count changes after each intervention type. The parameter vector θ governs:

- **Baseline step increase:** Expected Δsteps when no intervention is sent
- **Intervention effect size:** Marginal Δsteps per action type, modelled as $\Delta_{\text{effect}}(a_t) \sim \mathcal{N}(\mu_a, \sigma_a^2)$
- **Fatigue sensitivity:** Decay rate δ such that response probability $p(\text{respond}) = \max(0, 1 - \delta \cdot \text{burden})$
- **Burden threshold:** B_{max} , the number of consecutive interventions before response probability reaches zero

Estimation procedure:

1. Fit baseline step distribution from non-intervention control periods
2. Compute empirical effect sizes per action type from intervention windows
3. Cluster users by response pattern to assign archetype labels
4. Estimate archetype-specific parameters via maximum likelihood on held-out users

In Phase 1, synthetic parameter values are drawn from published statistics (mean step counts by age and gender, published HeartSteps engagement curves) [2, 3]. Real MRT data calibration is Phase 2.

7.2 Calibration Loop

After parameter estimation, the simulator is calibrated by comparing population-level statistics:

1. Generate N synthetic user trajectories from the simulator
2. Compute target statistics from real data (or published benchmarks): mean daily steps, response rate by week, burden accumulation rate
3. Adjust θ to minimise discrepancy between simulated and target statistics
4. Repeat until convergence or budget exhausted

7.3 Fidelity Metrics

Response rate RMSE

Root mean squared error between simulated and real response rates, bucketed by week and action type. Target: < 0.05 .

Step count distribution KL divergence

KL divergence between simulated and real daily step distributions. Target: < 0.1 nats.

Engagement decay alignment

Slope of engagement-over-time curve. Target: within 20% of real decay rate.

Archetype separation

Do distinct archetypes produce distinct reward trajectories? Measured by inter-archetype mean reward variance.

7.4 Limitations

The rule-based simulators in Phase 1 capture first-order effects only:

- No habituation or context-dependent preference shifts
- No social or environmental confounders (weather, holidays)
- No long-term habit formation (positive reinforcement compound effects)

These limitations motivate Phase 2 LLM-based simulation.

8 Data Sources

This project targets two tiers of data: synthetic for Phase 1 development and real-world for Phase 2 validation.

8.1 Phase 1: Synthetic Data

All Phase 1 experiments use synthetic data generators parameterised from published statistics. This avoids the 4-8 week lead time for real dataset access. The **SyntheticDataGenerator** produces realistic wearable traces by sampling from population distributions reported in the literature (e.g., NHANES step counts by age/gender [4], All of Us sleep distributions [5]).

8.2 Phase 2: Real Data Targets

1. **All of Us Fitbit Dataset** [5]: 59,000+ participants, Fitbit data, 14-year span. Access via cloud workbench only.

2. **UK Biobank Accelerometer Dataset** [6]: 700,000+ person-days, wrist-worn tri-axial accelerometer. Requires institutional application (4-8 weeks).
3. **HeartSteps V1/V2**: Micro-randomized trials with intervention response data. Directly calibrates the user simulation (1C). Access requires data sharing agreement (weeks to months).

8.3 Dataset Survey

See `docs/sources/data_sources.md` for the full feasibility study (All of Us, UK Biobank) and `docs/sources/additional_data_sources.md` for a survey of 12 additional datasets including HeartSteps, WISDM, ExtraSensory, and open-access coaching intervention datasets. Key findings:

- Between All of Us and UK Biobank, most wearable MDP variables (steps, HR, sleep, sedentary, time-of-day) are covered.
- Gaps are simulated variables: goal progress, burden, user archetype.
- Body measures need EHR linkage (All of Us has 46% linked).
- HeartSteps is the only available source of action-to-response mappings for direct simulator calibration.

9 Timeline & Future Work

9.1 Phase 1 Milestones

Table 5: Phase 1 milestones

Week	Milestone
1–2	Data layer (1A) + MDP environment (1B)
3–4	User simulation (1C) + Agent library (1D)
5–6	Experiment runner (1E) + end-to-end tests
7–8	Documentation, examples, handover

9.2 Phase 2: Future Work

- LLM-based user simulation (prompt-driven persona modelling)
- Offline RL agents (CQL, IQL) for batch policy learning
- Non-stationary environment adaptation
- Cross-dataset transfer experiments
- Real-data validation pipeline (All of Us, UK Biobank)
- Integration with real-world JITAI

References

- [1] A. Karine and B. M. Marlin. StepCountJITAI: A simulation environment for reinforcement learning with application to physical activity adaptive interventions, 2024. NeurIPS 2024 Workshop on Behavioral ML.
- [2] P. Klasnja, S. Smith, N. J. Seewald, A. Lee, et al. Efficacy of contextually tailored suggestions for physical activity: A micro-randomized trial. *Annals of Behavioral Medicine*, 53(6):573–583, 2019.

- [3] P. Klasnja et al. A RL-enabled micro-randomized trial for physical activity promotion. *npj Digital Medicine*, 2022.
- [4] PhysioNet. Minute-level step count from NHANES, 2025.
- [5] C. A. Patten et al. All of Us Fitbit Dataset: Large-scale wearable device data for health research. *Nature Medicine*, 2026.
- [6] L. Williamson et al. Self-supervised learning for wearable accelerometer data: The UK Biobank study. *npj Digital Medicine*, 7, 2024.